

ADA Quiz 2 Solutions

Supratim Shit, Syamantak Das, Divyajeet Singh, Aditya Upadhyay

March 2025

Solution 1.

(a) Graph Modeling (3 marks)

We model the sewer network as an undirected weighted graph $G = (V, E)$ with the following definitions:

- **Vertices** (V) : Each junction in the network is represented by a vertex.
- **Edges** (E) : Each pipe connecting two junctions is represented by an edge (u, v) with an associated weight w_{uv} that denotes the pipe's thickness.
- **Starting Vertex** : The manhole where Aquaman enters the network is the starting vertex s .

Aquaman is able to traverse an edge (u, v) if and only if the corresponding weight satisfies $w_{uv} \geq \lambda$ where λ is Aquaman's breadth.

Grading

The following marks will be awarded separately for mentioning each of the following.

- 1 mark for mentioning the correct vertex set V .
- 1 mark for correctly identifying weighted edges E (or edges and a weight function w). 0 marks if edge weights are not mentioned.
- 0.5 marks for identifying the start vertex s .
- 0.5 marks for identifying λ as a constraint.

(b) Formal Problem Definition (2 marks)

Formulating as a graph problem, we are given

- An undirected weighted graph $G = (V, E)$ with a function $w : E \rightarrow \mathbb{Z}^+$ (or simply weighted edges)
- A starting vertex $s \in V$
- A threshold value λ denoting Aquaman's breadth.

Objective: Determine the set $R \subseteq V$ of vertices such that for every vertex $v \in R$, there exists a path from s to v consisting exclusively of edges (u, v) where $w_{uv} \geq \lambda$.

Grading

The following marks will be awarded for mentioning the following.

- 1 mark for identifying as a *vertex reachability* problem. **Note:** the words 'vertex reachability' are not required - but the idea that we require vertices which are reachable from s with some constraint should be clear.
- 1 mark for mentioning the weight constraint $w_e \geq \lambda$.

(c) Pseudocode in Adjacency List Representation (5 marks)

Algorithm 1 gives the pseudocode using Breadth-First Search (BFS), which is well-suited given the connectivity constraint and the limited degree of each vertex.

Algorithm 1 Find reachable junctions with edge weight constraint

Require: Graph G as an adjacency list, number of vertices n , starting vertex s , threshold λ

Ensure: Set R of reachable vertices and their minimum distances

```
1: distance[v]  $\leftarrow \infty \quad \forall v \in V$ 
2: distance[s]  $\leftarrow 0$ 
3: visited  $\leftarrow \{s\}$ 
4: queue  $\leftarrow$  empty queue
5: Enqueue ( $s, 0$ ) into queue
6: while queue is not empty do
7:   ( $u, d$ )  $\leftarrow$  queue.dequeue()
8:   for each neighbor ( $v, w$ ) in  $G[u]$  do
9:     if  $v \notin \text{visited}$  and  $w \geq \lambda$  then
10:      Add  $v$  to visited
11:      distance[v]  $\leftarrow d + 1$ 
12:      Enqueue ( $v, d + 1$ ) into queue
13:     end if
14:   end for
15: end while
16: return distance
```

$\triangleright$ distance[v] = ∞ means vertex v is unreachable from s

Note to TAs: There can be multiple correct answers. For example, a DFS-based solution can also help determine reachability (though it does not give the minimum distance (which was not required)). In such a case, vertices in R can be *marked* or added to a separate set.

Grading

- (Full marks) 5 marks for fully correct DFS/BFS-based algorithm using an adjacency list.
- (Full marks) 5 marks for any fully correct algorithm using *any valid* graph representation like adjacency matrix or sets V and E ONLY IF the overall time complexity of the solution is $O(n)$.
- (Partial marks) Partial marks for a *partially correct* pseudocode, as long as the idea of what it is trying to do is clear.
- (No marks) 0 marks if Dijkstra-like approaches are used as time complexity will be higher.

(d) Runtime Justification (2 marks)

Since each junction is connected to at most 4 pipes, the degree of any vertex u satisfies:

$$\deg(u) \leq 4.$$

Thus, the total sum of the degrees over all vertices is:

$$\sum_{u \in V} \deg(u) \leq 4n.$$

By the Handshake Lemma,

$$2|E| = \sum_{u \in V} \deg(u) \implies |E| \leq 2n \implies |E| = O(n)$$

In the algorithm, each vertex is enqueued and processed at most once, leading to $O(|V|) = O(n)$ time. Each edge is also examined at most once. Since $|E| \leq 2n$, this step takes $O(|E|) = O(n)$ time. Note that this is because our algorithm is just a BFS (a constrained one at that, so we may never see some vertices/edges). Thus, the overall time complexity of the algorithm is $O(|V| + |E|) = O(n)$.

Grading

- (No marks) 0 marks for mentioning $O(n)$ without any explanation.
- (No marks) 0 marks If the bound on $|E|$ is not shown. Sorry, an explanation is always required. :(
- Otherwise, mark partially using
 - 1 mark for identifying the correct bound on $|E|$ (naming *Handshake lemma* is not required, we need the bound) **OR** 0.5 marks for partially incorrect bound **OR** 0 marks for a meaningless bound or no explanation of why $|E| = O(n)$.
 - 1 mark for mentioning that seeing all vertices at most once and all edges at most one are both $O(n)$ **OR** 1 mark for mentioning that BFS/DFS runs in $O(|V| + |E|) = O(n)$ here.

Solution 2.

- Minimum number of SCCs: n
- Maximum number of SCCs: n

Grading

For both parts, use the following scheme

- (No marks) 0 marks if only n is written as the min/max number of SCCs without any explanation. Sorry :(
- Otherwise, 0.5 marks for correct min/max number of SCCs, and 0.5 marks for explanation (a one line explanation works).

Proof (No marks assigned for proof)

This proof is just for reference for students.

Proof by Contradiction

Proof. Assume that there exists a DAG $G = (V, E)$ with n vertices that does not have exactly n SCCs. Then, there must exist an SCC C that contains at least two distinct vertices, say u and v (i.e., $|C| \geq 2$). By definition of a strongly connected component, there exists a path from u to v and a path from v to u . Combining these paths forms a cycle. However, by the definition of a DAG, no cycles are allowed. This contradiction implies that our assumption is false. $\square$

Solution 3.

Part (a) (3 marks)

FALSE - no, this version of Kosaraju's algorithm does not work.

Counterexample (sorry, I didn't have time to make it pretty):

$$G = v_1 \leftrightarrow v_2 \leftarrow v_3 \quad G^{rev} = v_1 \leftrightarrow v_2 \rightarrow v_3$$

(Normal Kosaraju) Start the DFS from v_1 in G^{rev} . Explore vertices in the order v_1, v_2, v_3 , and ordering the vertices in decreasing order of finish time gives $\{v_1, v_2, v_3\}$. If we now DFS from v_1 (highest finish time) in G , we identify the SCCs $\{v_1, v_2\}, \{v_3\}$. This is the correct answer.

(Modified Kosaraju) However, now consider the vertices in the opposite (non-decreasing/increasing) order of finish times, i.e. $\{v_3, v_2, v_1\}$. Start DFS from v_3 in G . This results in identifying $\{v_3, v_2, v_1\}$ as one SCC, which is **incorrect**.

Grading

- (No marks) 0 marks if only **FALSE** is written without any explanation. Sorry :(
- Otherwise, 1 mark for recognizing **FALSE**, 1 mark for a correct counterexample, and 1 mark for explanation. Again, even a correct one-line explanation works.

Note to TAs: There can be *a lot of* counterexamples to this. DO NOT match their graphs to mine, and verify that the provided graph indeed gives an incorrect answer using this modified Kosaraju's considering vertices in the increasing order of finish times.

- If the counterexample is incorrect, then 1 mark for recognizing **FALSE** and 0.5 marks total for the counterexample + explanation.

Part (b) (3 marks)

TRUE - yes, this version of Kosaraju's algorithm works.

Proof. Assume that the original Kosaraju's algorithm works correctly on any directed graph $G = (V, E)$. A component C is strongly connected if and only if all vertices in C are reachable from some vertex $s \in C$ in both G and G^{rev} (this is the premise provided by Kosaraju's algorithm).

Useful Fact: We saw in the tutorial that a graph G is strongly connected if and only if its reversed graph G^{rev} is strongly connected. This extends to the fact that a component is strongly connected in G if and only if it is strongly connected in G^{rev} . So, to Identify the SCCs of G , it is sufficient to identify the SCCs of G^{rev} .

Apply Kosaraju's algorithm to G^{rev} . The first run of DFS is on the reverse of G^{rev} , i.e. on $(G^{rev})^{rev} = G$. The second run of DFS is on the input graph G^{rev} . Then, Kosaraju's algorithm correctly identifies whether G^{rev} is strongly connected. Using the aforementioned **useful fact**, we have correctly identified whether G is strongly connected.

Finally, note that $\text{KOSARAJU}(G^{rev})$ is equivalent to the given $\text{KOSARAJU-MODIFIED}(G)$. □

Grading

- 0 marks if only **TRUE** is written without any explanation. :(
- Otherwise, 1 mark for recognizing **TRUE** and 2 marks for a correct proof. The proof does not need to be super formal, but should precisely convey the idea. Partial marks should be awarded for partially correct proofs.

Note to TAs: Again, there are multiple ways of proving this. Please award marks for any logically sound argument that proves this modified Kosaraju's works.

- If a *proof by example* is used (i.e. giving an example to say that the statement is true), then 1 mark for recognizing **TRUE** and 0 marks for the 'proof'.

Note to Students: A *proof by example* is **not a valid proving technique**. Just because something holds for your example input does not imply it holds for every input.